

echo & 变量

So simple, yet so complex

NAME: echo

- write arguments to the standard output
- 将输入参数写入标准输出（屏幕）



echo

Hello World!

● `$ echo Hello World!`

- 不需要定义主函数
- 不需要变量类型
- 不需要引入头文件
- 不需要指定参数
- 不需要预先安装SDK
- 没有源代码文件
- 可执行文件
- 没有编译、连接
-



SHELL 编程与执行指令有区别吗?

SHELL 脚本

- 脚本中保存了一系列指令，执行脚本就是**按顺序**执行脚本中的所有指令
- **SHELL** 有很多命令，每一个命令的参数、选项不尽相同；
- 一个复杂操作会涉及到不同的指令，又需要根据上下文环境的不同选择不同的参数和选项
- 重复输入完整的指令机械、繁琐，可以将他们封装在一个可执行的脚本文件中，方便执行

为什么需要shell编程

- 快速、简单地利用既有程序实现功能
- 每个Unix/Linux都有一个Shell
- 程序任务相对简单，易于配置、维护和可移植
- 对执行效率要求较低

Hello World!

```
#!/bin/bash
# a demo for bash programming

echo hello world !
sleep 1|
```

hello.sh

Shebang

用于指定执行当前脚本的shell程序

行注释

开头的字符串，可在行内注释

Unix 指令

利用回车断行，来指定指令执行次序

通常脚本文件以 .sh 为类型名称

● \$./hello.sh

通过 ./ 来执行这个脚本

指令 echo

- `$ echo #display a line of text`
 - 将输入参数输出在屏幕上，并默认加上一个换行符"`\n`"
- `$ echo hello world`
- `$ echo -n`
 - `-n`: 不在输出最后添加换行符

其它echo参数 (in GNU echo)

- **-E**: 使转义字符失效 (默认如此)
- **-e**: 使转义字符生效

No more ...

echo 转义字符

转义字符	字符的意义
\a	ALERT / BELL(从系统的喇叭送出铃声)
\b	BACKSPACE, 也就是向左退格键
\c	取消行末之换行符号
\E	ESCAPE, 脱字符键
\f	FORMFEED, 换页字符
\n	NEWLINE, 换行字符
\r	RETURN, 回车键
\t	TAB, 表格跳位键
\v	VERTICAL TAB, 垂直表格跳位键
\n	ASCII 八进制编码(以x开头的十六进制和以0开头的十进制), 此处的n为数字
\\	反斜杠本身

echo examples with -e

- `$ echo -e "a\tb\tc\nd\te\tf"`
- `$ echo -e "\0141\011\0142\011\0143\012\0144\011\0145\011\0146"`
- `\0141: "a" in ASCII; \011: "\t" in ASCII`

echo examples with -e

- `$ echo -e`
`"\x61\x09\x62\x09\x63\x0a\x64\x09\x65\x09\x66"`
- ASCII in Hex
- `$ echo -e "a\tb\tc\nd\te\bf\a"`
- `\b`: Backspace
- `\a`: 从系统的喇叭送出铃声

echo 变量

- `$ echo $HOSTNAME`
- `$ echo ${USER}`
- `$ echo $?`
 - 获取最后一条指令的返回状态，一般"0"表示指令正常退出，而非"0"则表示异常
- `$ touch a ; ls a 1>/dev/null; echo $?`
- `$ rm b 2>/dev/null; ls b 1>/dev/null 2>1; echo $?`

SHELL 变量

- `$ A=B` # 设定一个变量A，其值为B
- `=` 两边不可有**IFS**，并尽量避免包含**meta**
 - `IFS: <Space>, <Tab>, <Enter>`
- 变量名不可用\$符
- 变量首字母不可是数字
- 变量长度不可超过256字母
- 变量与值是大小写敏感的

查看 IFS 字符

- `$ echo "$IFS"`
- `$ echo -n "$IFS"`
- `$ echo -n "$IFS" | od -c`

```
$ echo $IFS | od -c
```

```
$ echo "$IFS" | od -c
```

比较一下这两个命令，想想为什么？

变量定义不会传递

● \$ A=B

● \$ B=C

● \$ echo \$A

B

● \$ B=\$A

● \$ A=C

● \$ echo \$B

B

取消变量定义

- `$ unset variable_name`

```
$ A=B
$ echo $A
$ unset A
$ echo $A

$ export A=B
$ echo $A
$ unset A
$ echo $A
```


执行SHELL脚本 & 环境变量

./

source

exec

~~执行程序就是双击~~



执行shell脚本

- **Ubuntu** 系统下，默认文件权限为**666**，默认掩码 **umask** 为**022**，因此新建文件权限为**644**，不可执行
- 因此，需要首先赋予脚本可执行的权限
 - `$ chmod +x ***.sh`

执行shell脚本

- 执行shell脚本的三种方式

1. `$ ./***.sh` # 执行当前文件夹内的脚本

2. `$ /.../.../... ***/***.sh` # 通过绝对路径定位并执行脚本

3. `$ ***.sh` # 直接执行，需要脚本所在的路径

列于环境变量：可搜索路径 `$PATH` 中

`$ echo $PATH`

环境变量

- 由`shell`维护的一组变量，用于记录特定的系统信息
 - 系统名称、登录用户名、用户ID、主目录、搜索路径等
- `$ env`
 - # 查看当前`shell`中的全局和局部环境变量，无法区分

环境变量

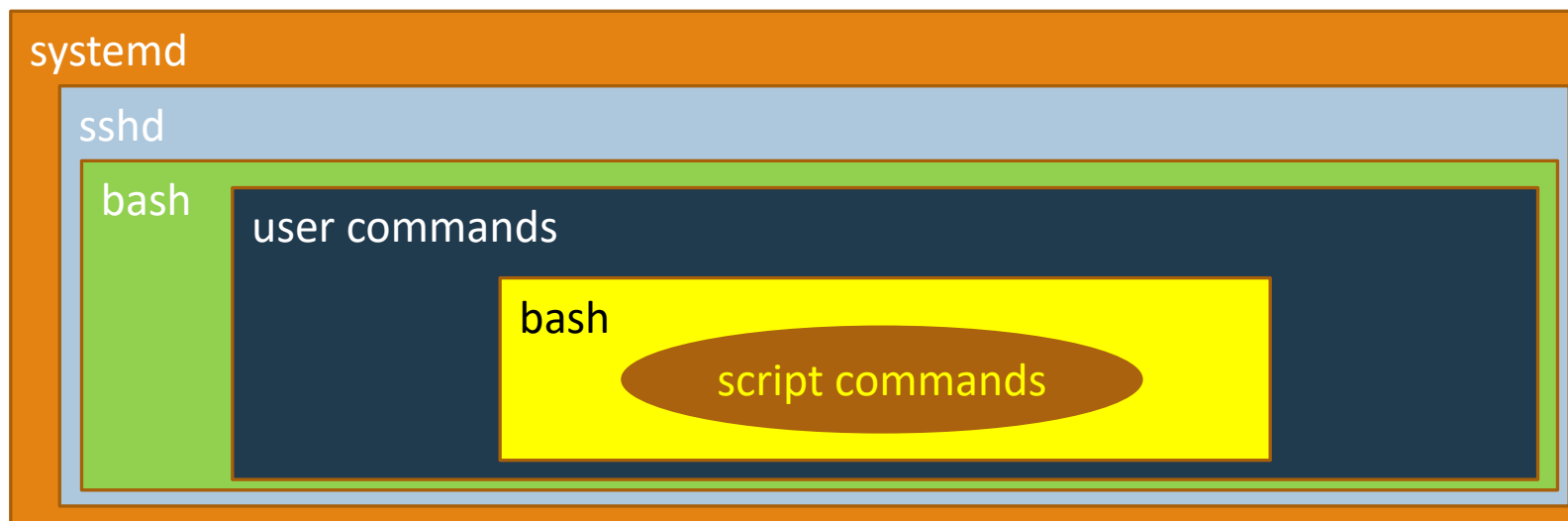
- 利用 `export` 命令来进行定义单个环境变量
- 引用时采用`$VAR`，或者`${VAR}`的方式进行引用
- 环境变量值可以定义多个，由“:”间隔
 - `HOME` 表示用户的主目录在文件系统中的位置。
 - `TERM` 指定使用终端或虚拟终端的类型
 - `USER` 表示当前用户名称
 - `PATH` 罗列出用户输入命令的搜索路径。
 - `LANG` 表示当前系统的默认语言与编码

环境变量

- 环境变量的生命周期
 - 进程在创建时继承其父进程的绝大部分环境设置
- 环境变量分为
 - 全局环境变量：适用于所有的`shell`会话和衍生的子`shell`
 - 局部环境变量：只能在定义它们的`shell`进程中可见
- 在何时定义全局和局部环境变量？

SHELL脚本的执行过程

- 在Shell中执行脚本，其基本过程是：
 - 在当前shell进程中fork一个shell子进程，用于执行对应程序
 - 程序结束后，再返回当前进程（父进程）。



SHELL命令的执行过程

- 下列指令会发生什么？

- `$ echo "cd $HOME/.ssh" > goto.sh`

- `$ chmod +x goto.sh`

- `$ ./goto.sh`

Tip!

PWD 是一个环境变量



SHELL命令的执行过程

- `$ ./*.sh` # 默认的, 以`fork`方式执行脚本
- `$ source *.sh` # 在当前进程执行脚本
- `$ exec ./*.sh` # 终止当前进程, 并在其中执行新程序

SHELL命令的执行过程

~/demo/ch05/forkShell/1.sh

#!/bin/bash

```
A=B
echo "PID for 1.sh before exec/source/fork:$$"
export A
echo "1.sh: \ $A is $A"
case $1 in
  exec)
    echo "using exec..."
    exec ./2.sh ;;
  source)
    echo "using source..."
    source 2.sh ;;
  fork)
    echo "using fork by default..."
    ./2.sh ;;
  esac

echo "PID for 1.sh after exec/source/fork:$$"
echo "1.sh: \ $A is $A"
```

~/demo/ch05/forkShell/2.sh

#!/bin/bash

```
echo "PID for 2.sh: $$"
echo "2.sh get \ $A=$A from 1.sh"
A=C
export A
echo "2.sh: \ $A is $A"
```

SHELL 模式

/etc/profile

/etc/bash.bashrc

~/.bash_profile

~/.bash_login

~/.profile

~/.bashrc

~/.bash_logout

- 用户退出登录操作：
 - logout
 - exit
 - <Ctrl-D> (end-of-file 信号)

logout 是注销，
exit也是注销。
它们有什么区别？



SHELL 模式

- `shell` 是一个程序

- `$ file $SHELL`

不同的 SHELL 模式下，所需要的环境变量，以及环境变量的内容都可能存在差异。

- `shell` 程序可以充当用户界面：交互式

- 打开一个`shell`，输入命令，获得反馈

- `shell` 程序也可以独立于人来运行：非交互式

- 获得一个文件，对文件中的内容（`shell`指令）进行解释和执行

SHELL 模式

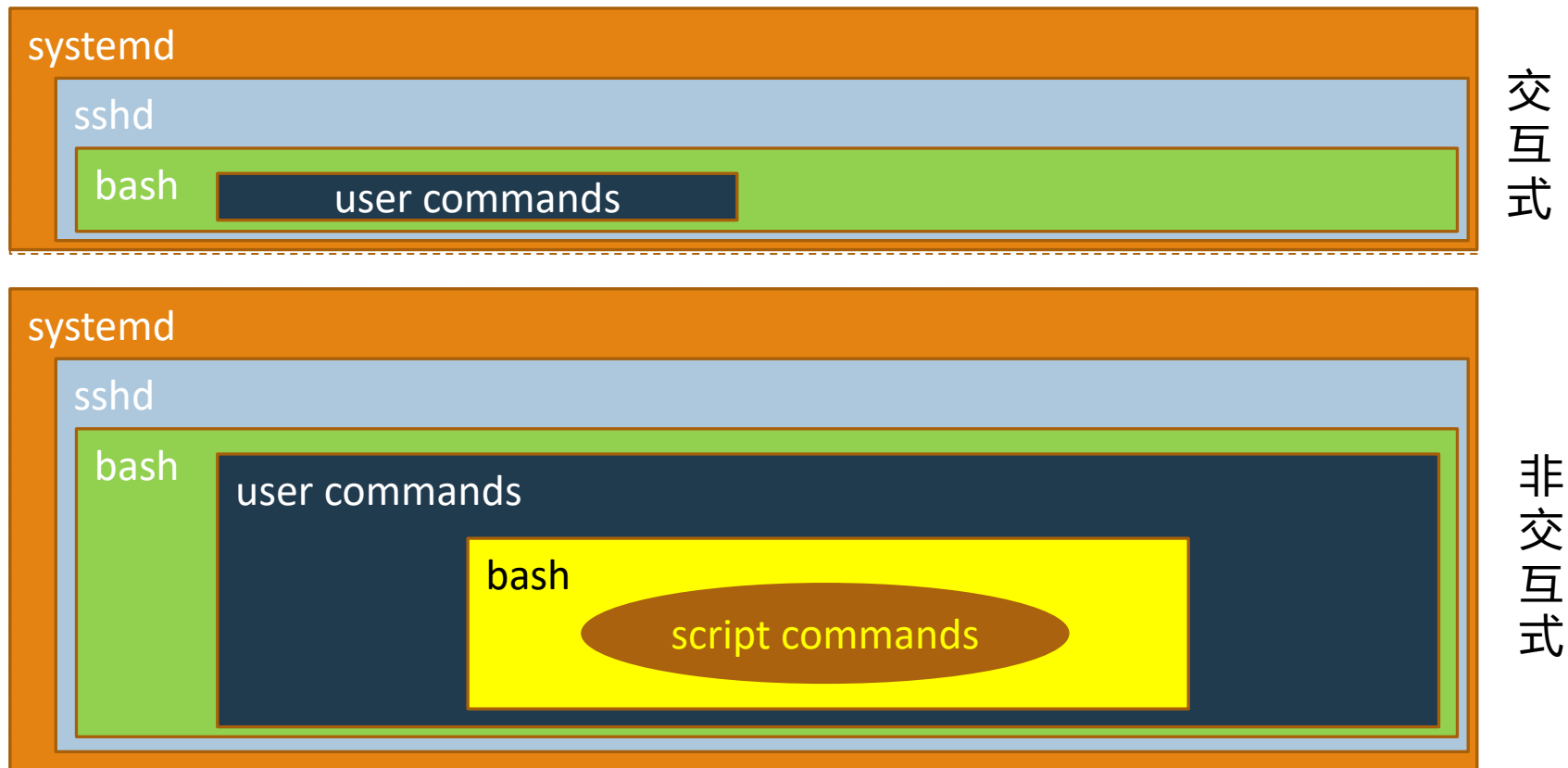
- `$ pstree -p | head -n 1 ; pstree -p | grep $$`

交互式

- `$ echo 'pstree -p | head -n 1 ; pstree -p | grep $$' > shell_mode.sh`
- `$ chmod +x shell_mode.sh`
- `$ ./shell_mode.sh`

非交互式

SHELL 模式



SHELL 模式

SHELL 模式		登入方式	退出方式
交互 SHELL	登录SHELL	<code>bash --login</code> <code>su - / su -l</code>	<code>logout /</code> <code>exit</code>
	非登录SHELL	<code>bash</code>	<code>exit</code>
非交互SHELL		执行脚本文件	<code>exit</code>

- 登录SHELL时与非登录SHELL时所使用的初始化文件不同
 - `/etc/profile`、`~/.profile`、`~/.bashrc`
 - 来尝试一下?
- 非交互SHELL的初始化文件由环境变量`BASH_ENV`决定

初始化和注销文件

- 初始化文件
 - 环境文件 (`environment file`)
 - 登录文件 (`login file`)
- 注销文件 (`logout file`) : 注销时自动执行的文件
 - `~/.bash_logout`

初始化文件中的内容

- **环境文件**：在新`shell`启动时自动执行的命令
 - 设置不能在环境中保存的自定义项，如`shell`选项，别名，`SHELL`函数等，因为它们不能被继承，每次启动新`shell`都必须重新创建
- **登录文件**：基于环境文件，补充在登录时才需要自动执行的命令
 - 设置环境以及初始化工作会话
 - 创建或者修改环境变量，它们会被所有的子进程（包括子`shell`）所继承
 - 执行所有的一次性操作

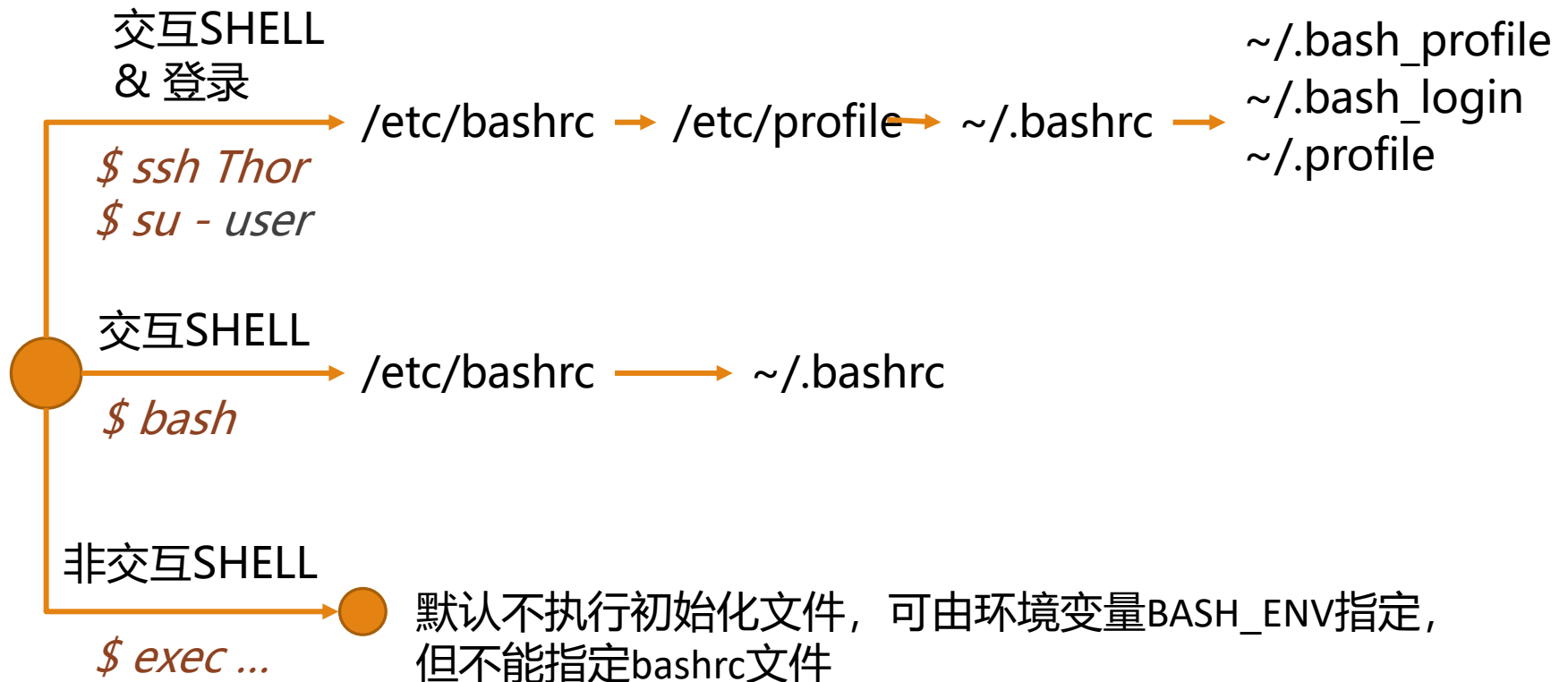
环境文件 (Bash in Ubuntu)

- 全局（公用）配置文件
 - `/etc/bash.bashrc` （一般为 `/etc/bashrc`）
- 本地配置文件
 - `~/.bashrc`
 - `rc` 代表 “run commands”，指自动运行的命令

登录文件 (Bash in Ubuntu)

- 全局（公用）配置文件
 - `/etc/profile`
- 本地配置文件：依序读取第一个存在的文件
 1. `~/.bash_profile`
 2. `~/.bash_login`
 3. `~/.profile`

SHELL模式与初始化顺序



SHELL变量与环境变量

- 一般的SHELL变量属于local variable
- 经export 命令设置后, 属于environment variable
 - 只有环境变量才可以被子shell继承

```
$ A=B
$ export B=C
$ bash
$ echo $A
$ echo $B
$ echo -e '#!/bin/bash\n echo $B' > echoEVB.sh
$ chmod +x echoEVB.sh
$ ./echoEVB.sh
```

环境变量的设置

- `$ export A=B` #设置A为环境变量，并将其值赋为B
- `$ A=B ; export A` #设置变量A值为B，并将其设为环境变量
- `$ declare -x A=B` #设置A为环境变量，并将其值赋为B
- `$ unset A`

SHELL提示符

- 基本的SHELL提示符有两个（**Bash**下）：
 - \$ ：普通用户
 - #： `root` 用户

是否可以修改（个性化）SHELL提示符，使之更有用？



修改SHELL提示符

- Bourne shell中，修改\$PS1环境变量即可修改shell 提示符
- `$ export PS1=' ${USER} @ ${HOSTNAME} $'`
- 通过命令嵌套，来动态生成提示信息
- `$ export PS1=' $(date) $'`
- 利用转义字符的特殊码来增强提示
- `export PS1="\!_${USER}@${HOSTNAME} \w$"`
- 还可以带颜色显示 ([自学](#))

Bash 使用实例

- 通过修改 PS1 环境变量值, 可以更改shell提示
- `export PS1="\[\033[01;32m\]\u,`
`\[\e[1;31m\]I AM GROOT!!`
`\[\033[01;33m\]@[\$(date +%m-%d\" \"%H:%M) ]`
`\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$"`

标准输入输出

STDIN

STDOUT

STDERR

<, >, >>

- stdin (standard in): 标准输入
- stdout (standard out): 标准输出
- stderr (standard error): 标准错误



Linux 管道

- `pipeline` 是 UNIX 的传统进程通信方式
- 按照 **FIFO** 方式，在两个并发进程间进行数据传送
- 仅需在两个指令间插入 "`|`"，即可建立匿名管道



Linux I/O

- 输入和输出操作是每个程序的必备环节
- 导致大量重复代码，程序与数据文件高度耦合
- 不利于构建通用程序



[~/demo/ch05/pipe/readFile.cpp](#)

记得引用 fstream 库!

```
#include <iostream>
#include <fstream>
using namespace std;
```

只能打开 data 数据文件!

```
int main(int argc, char** argv){
    ifstream myFile;
    myFile.open("data");
    string line;
    while(getline(myFile, line)){
        cout << "Get a line: " << line << endl;
    }
    myFile.close();
    cerr << "Oops, it's the end !" << endl;
}
```

千万记得关闭文件!

Linux I/O

~/demo/ch05/pipe/readFile.cpp

```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  int main(int argc, char** argv){
6      ifstream myFile;
7      myFile.open("data");
8      string line;
9      while(getline(myFile, line)){
10         cout << "Get a line: " << line << endl;
11     }
12     myFile.close();
13     cerr << "\nOops, it's the end! " << endl;
14 }
```

~/demo/ch05/pipe/stdioe.cpp

```
#include <iostream>

using namespace std;

int main(int argc, char** argv){

    string line;
    while(getline(cin, line)){
        cout << "Get a line: " << line << endl;
    }

    cerr << "\nOops, it's the end ! " << endl;
}
```

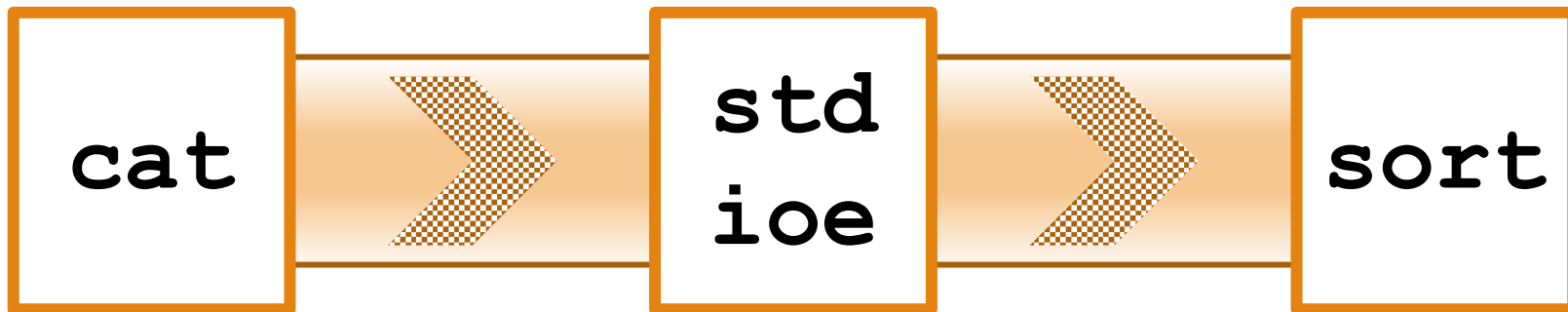
Linux 管道

- `cat` 指令用于在Linux 操作系统中读取文件内容
- `$ cat data | stdio`



Linux 管道

- `cat` 指令用于在Linux 操作系统中读取文件内容
- `$ cat data | stdioe | sort`



C++

- `cin >> var ;`
- `cout << var << endl;`
- `cerr << var << endl;`

Java

- `System.in.read();`
- `System.out.println();`
- `System.err.println();`



标准输入输出

- `stdin` (`standard in`): 标准输入
 - `cin`, `fread(stdin, ...)`
- `stdout` (`standard out`): 标准输出
 - `cout`, `fprintf(stdout, ...)`
- `stderr` (`standard error`): 标准错误
 - `cerr`, `fprintf(stderr, ...)`

重定向符

- < : 标准输入重定向
- > : 标准输出 (**stdout**) 重定向
 - 若文件不存在, 则新建 \$./stdioe > result
 - 若文件已存在, 则覆盖! \$./stdioe < stdioe.cpp > result
- >> : 标准输出 (**stdout**) 重定向
 - 若文件不存在, 则新建
 - 若文件已存在, 则追加

如何区分 **STDOUT** 和 **STDERR**?

重定向符

- 文件描述符 (`file descriptor`) : 文件的数字标识
 - `STDIN: 0`
 - `STDOUT: 1`
 - `STDERR: 2`
- 重定向符的正式语法是在符号前使用文件描述符
 - `0< ; 1> ; 2>>`
 - `$ ./stdioe 0< stdioe.cpp 1> result 2>> error`

重定向符

- `echo` 不同类别消息至 `stdout` 和 `stderr`
 - `$ cd ~/demo/ch05`
 - `$ ./redirect.sh`
 - `$ ./redirect.sh >/dev/null`

组合STDOUT & STDERR

- `$ command x>outputfile y>&x`
 - `x` 输出到`outputfile`, 然后`y`输出到`x`
 - `$ ./stdioe 0< stdioe.cpp 1> result 2>&1`
- 注意! `$ command y>&x x> outputfile` 无效!
 - 在处理 `y>&x` 时, `x`仍输出向原始位置
 - 然后`x`才更改了输出位置

组合STDOUT & STDERR

- 在 Bourne shell 家族中, 可以采用下述方法
 - `$ ./stdioe2 0< stdioe2.cpp &> result`
- 如果要追加, 那么只能
 - `$ ./stdioe2 0< stdioe2.cpp >> result 2>&1`

抛弃输出

- `$ ./stdioe2 0< stdioe2.cpp >/dev/null`
 - 只显示错误信息
- `$ ./stdioe2 0< stdioe2.cpp 2>/dev/null`
 - 忽略错误信息
- `/dev/null` 是一个伪设备文件, `/dev/zero`类似。
任何写入这两种设备上的输出会被抛弃

Shell 编程基础环境

\$./* .sh {

- fork (default)
- source
- exec

*.sh

#!/bin/bash

Shebang

echo "hello world! "

Unix 指令

Shell 变量

- 普通变量 / 环境变量
- 变量命名、赋值、引用
- 数组变量, 变量计算

STDIN



0<

|

STDOUT



1>

|

STDERR



2>



[pure-bash-bible](#): 📖 A collection of pure bash alternatives to external processes